

Übungen zur **Computergraphik I** (SS 2026)

Blatt Nr. 7, Abgabe: Dienstag, 9. Jun, 12:00

Hinweise

- Die bearbeiteten Übungsblätter müssen jeweils Dienstag **vor** der ersten Übung im ELearning Portal eingestellt sein (**Einzelaufgaben**).
- Übungsblätter und Hilfsmittel sind hier zu finden (Passwort:):
<https://elearning.uni-bayreuth.de/course/view.php?id=48309>
- Schriftliche Abgaben bitte als **PDF** dem Archiv hinzufügen (es gibt **keine** Papierabgaben)
- Für eine einzelne Datei wie **PDF** laden Sie sie einfach direkt hoch. Wenn es sich um mehrere Dateien handelt, bitte legen Sie alle Dateien (auch PDF...) vor dem Archivieren in einen Ordner namens **UEnn**.
- Die Lösungen bitte in einem Archiv (**Zip oder 7z**) hochladen.
Dateinamen-Format: **UEnn_Ihr_Name.zip** (oder .7z, .pdf)
- Abgaben, die nicht kompilieren, Shader oder C++, werden mit 0 Punkten bewertet.

Aufgabe 17 Anti-Aliasing und Culling (*Theorie*)

9 Punkte

- a) Was sind die Unterschiede zwischen Multisampling (MSAA) und Supersampling (SSAA), und welches Verfahren wird häufiger verwendet? **[3P]**
- b) Eines der Kernprobleme des Painter-Algorithmus ist die Unfähigkeit, sogenannte „zyklische Überlappungen“ (Cyclic Overlaps) korrekt aufzulösen. Beschreiben Sie in einfachen Worten, was unter einer zyklischen Überlappung zu verstehen ist. Mit welchen zwei grundlegenden Ansätzen bzw. Algorithmen der Computergrafik lässt sich dieses Problem effektiv lösen? **[3P]**
- c) Beschreiben Sie den Ablauf des Z-Buffer-Algorithmus kurz in Form von Stichpunkten. **[3P]**

Aufgabe 18 QuadTree (*Praxis*)

6 Punkte

In der Vorlesung haben Sie sich mit *BSP-Bäumen*, *Octrees* beschäftigt. Anders als in der Vorlesung sollen Sie in dieser Aufgabe über eine bestehende Geometrie einen Quadtree bauen und anzeigen (im Ordner exo6). Hierfür machen Sie bitte folgendes:

- a) Hüllkörper Axis Aligned Bounding Box (AABB): **(1 Punkte)**

- Implementieren Sie den Konstruktor mit vernünftigen Initialwerten
- Implementieren Sie die Funktion `addPoint`, welche den Hüllkörper um den neuen Punkt erweitert. Bauen Sie den Baum nur bis zu einer maximalen Tiefe auf. Nutzen Sie hierfür die Variable `max_depth`

b) QuadTree: **(2 Punkte)**

- Implementieren Sie die Funktion `insertPoint` rekursiv, welche einen neues Objekt (Vertex) in den QuadTree einfügt. Die Daten sollen dabei in den Blattknoten des Baumes gespeichert werden.
- Implementieren Sie die Funktion `generateVertexList`, welche eine Liste der Vertexdaten zurück gibt, mit denen der QuadTree gezeichnet werden kann. Hierbei soll jede Tiefe des Baumes eine unterschiedliche Farbe erhalten.

c) Visualisierung des Baumes: **(3 Punkte)**

Es soll sowohl der Baum als auch die Objekte angezeigt werden. Die verschiedenen Level des Baumes sollen in unterschiedlichen Farben dargestellt und die Objekte rot eingezeichnet werden.

- Erzeugen und Löschen der benötigten Buffer.
- Implementierung der Vertex- und Fragment-Shader.
- Anzeige der Daten.

(Hinweis: Machen Sie sich mit den verschiedenen Primitiven in OpenGL vertraut.)